GenericAgent: A Context-Density-Driven LLM Agent System for Long-Term Autonomous Operation

Abstract

Autonomous agents built on large language models face a fundamental bottleneck: as tasks extend over long horizons, redundant and low-value information accumulates within the finite context window, progressively degrading reasoning quality and decision accuracy. This problem demands a principled approach to managing what information occupies the context at each reasoning step. We present GenericAgent (GA), a general-purpose LLM agent system designed around the principle of context information density maximization, supporting three complementary operating modes—Interactive, Task, and Reflect—for sustained autonomous operation. GA makes four main contributions: (1) an L0–L3 hierarchical memory architecture that loads information on demand and validates updates through action-level verification; (2) a reflection-driven self-evolution mechanism that progressively compresses experiential knowledge from natural-language experience to structured SOPs to executable code and reusable skills; (3) a minimal set of seven atomic tools whose principled composition achieves full environmental control; and (4) a simplified HTML browser control scheme with incremental DOM monitoring for efficient web interaction. We release GenericAgent as an open-source system to facilitate research on long-term autonomous agents.

1 Introduction

Large language model (LLM) based autonomous agents have advanced rapidly in recent years, demonstrating remarkable capabilities in code generation, web browsing, and complex task automation [1–3]. Systems such as Claude Code [4], Codex [5], and OpenHands [6] can now interpret high-level instructions and execute multi-step plans that span dozens of tool invocations. Yet the vast majority of these systems are architected for single-session task completion: an agent receives a request, carries it out, and terminates without retaining any knowledge for future sessions. This design assumption breaks down when agents are expected to operate autonomously over extended time horizons—managing personal workflows, monitoring evolving environments, or accumulating domain expertise across hundreds of interactions. In such long-term settings, the context window becomes the fundamental bottleneck rather than model capability itself [7]. Tool definitions, historical memory, retrieved documents, and raw web content all compete for the same finite token budget, continuously expanding the prompt while squeezing the space available for effective reasoning. Empirical evidence consistently shows that, given the same informational requirements, longer prompts degrade model performance: attention becomes diluted, irrelevant tokens introduce noise, and critical instructions are more likely to be overlooked. This tension between the growing information demands of long-term operation and the fixed capacity of the context window constitutes the central challenge that motivates our work.

We identify four concrete dimensions along which context information density is eroded in existing agent systems, each representing a distinct source of what we term context pollution. First, **memory retrieval**: as an agent accumulates knowledge across sessions, its memory store grows without bound, yet each decision

typically requires only a small, precise subset of that knowledge—retrieving too much wastes context, while retrieving too little causes the agent to forget critical facts. Second, **experience reuse**: when an agent encounters a task similar to one it has solved before, the ideal behavior is to leverage prior experience; however, raw reasoning traces are verbose, filled with exploratory dead ends and redundant deliberation that are costly to re-inject into the prompt. Third, **tool overhead**: standard agent frameworks inject the full schema of every available tool into the system prompt on every LLM call, imposing a persistent, per-turn token cost that scales linearly with the size of the tool library [1]. Fourth, **web interaction**: when agents browse the web, raw HTML pages contain navigation bars, advertisements, scripts, and boilerplate that vastly outnumber the task-relevant content, yet conventional approaches pass this entire payload to the model. Existing systems have addressed these challenges in isolation—MemGPT [8] proposes OS-inspired virtual memory but lacks mechanisms for self-evolution, Voyager [9] introduces a skill library but is confined to Minecraft, and multi-agent architectures such as Manus [10] distribute complexity across specialized sub-agents without fundamentally reducing per-agent context consumption. No prior system has addressed all four dimensions under a unified design principle.

In this paper, we present **GenericAgent**, a general-purpose LLM agent system designed from the ground up around the principle of context information density maximization: within the finite context window available to each LLM call, the system should pack the highest-density, most task-relevant information so as to maximize the probability of task success while minimizing total prompt cost. The corollary of this principle is equally important: when the information required for a decision is already complete, a shorter prompt will yield better performance than a longer one, because every superfluous token is a source of distraction rather than signal. GenericAgent instantiates this principle through four tightly integrated subsystems—hierarchical memory, reflection-driven self-evolution, minimal atomic tools, and context-compressed browser control—each targeting one of the four information density challenges identified above. All subsystems are unified under a single Agent Loop that drives three complementary operating modes: **Interactive** mode for direct user commands, **Task** mode for sub-agent coordination via structured file I/O, and **Reflect** mode for event- or cron-triggered autonomous operation. The entire system is implemented in approximately 3,000 lines of core code, supports multiple LLM backends including Claude, OpenAI, and Gemini, and is publicly available at `https://github.com/lsdefine/GenericAgent`.

Our main contributions are as follows:

- A **hierarchical memory system** (L0–L3) with taxonomy-based indexing and action-verified updates, ensuring that only the minimal, validated information needed for each decision is retrieved into the context window.
- A **reflection-driven self-evolution** mechanism that progressively compresses raw experience into standard operating procedures, then into executable code, and ultimately into composable skills—each stage reducing the context cost of reusing past knowledge by an order of magnitude.
- A set of **seven minimal atomic tools** whose compact definitions occupy a fraction of the prompt budget consumed by conventional tool libraries, yet achieve full environmental control through principled composition [11].
- A **context-compressed browser control** scheme based on simplified HTML rendering and incremental DOM monitoring, which strips web pages down to their task-relevant semantic content before injection into the context window.

2 Related Work

2.1 LLM-Based Agent Systems

The development of LLM-based autonomous agents has progressed rapidly from prompting-level techniques to fully integrated production systems. Early work focused on augmenting a single LLM call with structured reasoning. ReAct [1] interleaves chain-of-thought reasoning [2] with environment actions, enabling models to ground their plans in observed feedback. Reflexion [11] extends this paradigm by introducing verbal self-reflection: after each trial the agent critiques its own trajectory and carries the critique forward, yielding substantial gains on sequential decision-making benchmarks.

Moving beyond single-turn prompting, AutoGPT [12] demonstrated that an LLM can autonomously decompose a high-level goal into sub-tasks, execute them through tool calls, and iterate until completion, sparking widespread interest in fully autonomous agents. MetaGPT [13] shifted the focus to multi-agent collaboration, encoding domain-specific Standard Operating Procedures (SOPs) into the communication protocol at design time so that specialized agents coordinate through structured message passing rather than free-form dialogue.

Software engineering has emerged as a primary proving ground. Devin [14], introduced as the first AI software engineer, couples long-horizon planning with an integrated development environment. SWE-agent [15] formalizes the notion of an Agent–Computer Interface (ACI), demonstrating that purpose-built commands for file navigation and editing significantly outperform raw shell access. OpenHands [6] provides an open-source platform unifying multiple agent architectures with a shared runtime, complemented by the OpenHands SDK [16] for rapid agent development.

Most recently, a wave of product-level systems has pushed the frontier of agent capabilities. Claude Code [4] adopts a single-threaded master loop with approximately 24 built-in tools, relying on compaction-based context management that triggers summarization when utilization reaches roughly 92% of the context window. OpenAI Codex [5] deploys an RL-optimized coding agent in isolated cloud sandboxes, emphasizing safe parallel execution. Manus [10] introduces a three-agent architecture—planner, executor, and verifier—built around the CodeAct paradigm in which executable Python code serves as the universal action format. OpenClaw [17] takes a messaging-platform-centric approach, centering agent interaction on a skills marketplace and attracting broad community adoption.

Despite this rapid progress, the vast majority of existing systems target single-session task completion or are tailored to a specific domain such as software engineering. Few address the challenge of long-term autonomy with a systematic optimization of context efficiency. GenericAgent is designed to fill precisely this gap.

2.2 Memory Systems for Agents

Effective memory management is critical for agents that must operate over extended horizons, yet existing approaches exhibit notable limitations. MemGPT [8] draws an analogy to operating-system virtual memory, maintaining a two-tier architecture in which the LLM's context window serves as "main memory" (RAM) and an external database acts as "disk." A paging mechanism swaps information between the two tiers on demand, but the approach inherits the complexity of page-fault handling without mechanisms to verify the integrity of what is stored.

A-MEM [18] takes inspiration from the Zettelkasten note-taking method, organizing memories as a dynamically evolving graph with semantic links between atomic notes. While the graph structure supports rich associative retrieval, the system lacks mechanisms to prevent hallucinated or stale information from contaminating the memory store over time.

Among production systems, Claude Code [4] persists memory as flat Markdown files and triggers compaction—a summarization pass that condenses the conversation—when context utilization approaches its ceiling. Manus [10] maintains a chronological event stream as working memory and offloads longer-term state to file-based artifacts such as `todo.md` for plan tracking. Both designs are predominantly append-only: new information is accumulated but rarely condensed, compressed, or actively verified against ground truth.

A common shortcoming across these systems is the absence of strict verification mechanisms that guard against hallucination contamination of the memory store. Updates are largely additive, and no system performs active condensation that distills accumulated knowledge into progressively denser representations. GenericAgent addresses these gaps through its L0–L3 hierarchical memory architecture and a "No Execution, No Memory" verification principle that conditions every memory update on an observed action outcome, ensuring that only empirically validated information enters the persistent store.

2.3 Self-Evolution and Experience Accumulation

A growing body of work investigates how agents can improve autonomously by accumulating and leveraging experience. Voyager [9] introduced the first LLM-based lifelong learning agent, constructing an ever-growing

library of executable skills within the Minecraft environment; each skill is a verified JavaScript program that the agent can compose to tackle increasingly complex tasks.

A recent survey on self-evolving agents [19] provides a comprehensive taxonomy along three axes: what to evolve (model parameters, memory, or tool repertoire), when to evolve (intra-test-time versus inter-test-time), and how to evolve (using scalar or textual feedback). Within this taxonomy, several concrete methods have been proposed. EvolveR [20] performs offline self-distillation of successful trajectories into strategic principles, which are then retrieved online to guide future decisions. FLEX [21] builds a structured experience library through continual reflection on both successes and failures, selecting relevant past experiences at inference time. AgentEvolver [22] represents experience as structured knowledge units expressed in natural language and injects them into the agent's context for in-context learning. The self-evolving lifelong learning framework [23] decomposes the evolution process into three stages—experience exploration, skill learning, and knowledge internalization—providing a principled progression from raw observations to reusable capabilities.

Despite their diversity, existing self-evolution approaches share a common limitation: they distill experience exclusively into textual guidelines—natural-language summaries, principles, or structured notes. None take the further step of codifying distilled experience into executable programs, thereby missing an opportunity for order-of-magnitude reductions in token cost. An SOP that requires hundreds of tokens to describe in prose can often be implemented as a short function consuming only a handful of tokens to invoke. GenericAgent's three-stage evolution pipeline—from natural-language experience to structured SOPs to executable code and reusable skills—bridges this gap, enabling progressively denser and more efficient representations of accumulated knowledge.

2.4 Tool Design and Context Engineering

Tool augmentation has become a standard technique for extending LLM capabilities. Toolformer [24] demonstrated that language models can learn to insert API calls into their own generations, while Gorilla [25] and ToolBench [26] address the challenge of selecting from large-scale tool repositories with hundreds or thousands of available APIs. Function calling interfaces, initially popularized alongside GPT-4 [3] and subsequently adopted across providers, now provide the dominant paradigm for structured tool invocation in production systems.

The Manus system [10] pushes this further with its CodeAct paradigm, in which executable Python code serves as a universal action format—subsuming individual tool calls within a more expressive programming abstraction. On the design-philosophy side, Anthropic's "Building Effective Agents" [27] and "Context Engineering" [7] documents offer the most systematic industry articulation of context efficiency principles to date, advocating system prompts written at the right level of abstraction, token-efficient tool definitions, just-in-time information retrieval, and aggressive compaction of historical context.

In practice, however, existing systems tend to address capability gaps by adding specialized tools: Claude Code [4] ships with approximately 24 built-in tools, and OpenHands [6] exposes a rich and extensible toolset. Each additional tool definition consumes context tokens for its schema, description, and usage examples, creating a tension between capability breadth and context efficiency.

Web browsing presents a particularly acute instance of this tension. Vision-based approaches such as Anthropic's Computer Use [28] operate on screenshot pixels, while DOM-based methods parse the page's HTML or accessibility tree. WebArena [29] and Mind2Web [30] have established standard benchmarks for evaluating web agents, revealing that both raw HTML and accessibility-tree representations contain substantial irrelevant information—navigation bars, decorative elements, tracking scripts—that dilutes the context with low-density tokens.

GenericAgent takes a fundamentally different approach. Rather than expanding the tool repertoire, it defines a minimal atomic set of only seven tools whose principled composition achieves full environmental control. Combined with DOM-level content compression that strips irrelevant markup before it enters the context window, this design simultaneously minimizes tool-definition overhead and maximizes the information density of web interaction observations.

[Figure 1: The Agent Loop architecture of GenericAgent]

Figure 1 Overview of the GenericAgent Agent Loop. At task start, global memory is loaded and merged with the system prompt. The LLM then enters a reasoning loop: if its response contains tool calls, each call is dispatched to the corresponding handler and the result is appended as a new user message for the next reasoning turn; if no tool call is present, the task terminates. After task completion, a memory distillation step optionally extracts valuable facts and experience into long-term storage.

3 GenericAgent

3.1 Overview of GenericAgent

GenericAgent (GA) is a general-purpose LLM agent system designed for long-term autonomous operation. Unlike conversational models that target single-turn task completion, GA is architected as a persistent, cross-session execution agent that accumulates experience over time and completes open-ended tasks with minimal user intervention. The system supports three complementary operating modes: **Interactive Mode**, in which GA responds to real-time user commands—file manipulation, code execution, information retrieval, and the like; **Task Mode**, in which GA acts as a sub-agent that receives delegated tasks from a parent agent, carries them out independently via a file-based I/O protocol, and returns structured results; and **Reflect Mode**, in which GA is triggered by external events or scheduled cron scripts to autonomously perform self-maintenance activities such as monitoring, summarization, and optimization, without any user-initiated request. All three modes share the same execution kernel, are driven by a unified Agent Loop, and support flexible switching among mainstream LLM backends including Claude, OpenAI, and Gemini.

The core capabilities of GA can be summarized along four dimensions. First, **structured memory management**: GA maintains a hierarchical, cross-session persistent memory system that stores environmental facts, task-level experience, and execution norms, enabling the agent to accumulate and reuse knowledge across sessions. Second, **tool-driven environmental interaction**: GA achieves comprehensive control over the file system, code execution environment, and web browser through seven atomic tools, complemented by a TMWebDriver component that performs JavaScript injection and page-state monitoring on the user's browser. Third, **experience self-evolution**: GA distills recurring task experience into standard operating procedures (SOPs), and further consolidates mature SOPs into directly callable code programs, progressively reducing the execution cost of similar tasks over time. Fourth, **autonomous reflection**: GA defines trigger conditions through programmable reflection scripts that automatically initiate tasks and record results when conditions are met, enabling sustained operation without human intervention.

The runtime flow of GA is centered on the Agent Loop, illustrated in Figure 1. At the start of each task, the system loads global memory from persistent storage, merges it with the system prompt, and constructs the session-level System Prompt. The user input is then combined with this System Prompt and submitted to the LLM for the first round of reasoning. If the LLM's response contains tool calls, the Agent Loop sequentially parses and dispatches each call to its corresponding tool handler; the tool's return value is appended to the conversation as a new user message, driving the next reasoning round. If the response contains no tool calls, the current task is deemed complete and the loop exits. A hard cap of 40 reasoning turns per task prevents runaway loops. As an additional safety measure, the system issues a warning against ineffective retries every 7 turns and forces a user confirmation via `ask_user` every 35 turns. Tool schemas are refreshed every 10 turns

to ensure consistency. Upon task completion, GA optionally triggers a memory distillation pipeline in which a dedicated sub-agent extracts valuable facts and experience from the current execution context and persists them to long-term memory. This flow is identical across all operating modes; the only differences lie in how tasks are sourced and how outputs are routed.

Every technical design decision in GA is governed by a single core principle: context information density maximization. The essence of this principle is to fill the finite context window with the highest-density content possible, ensuring task success at the minimal prompt cost. Given that the requisite information is already complete, a longer prompt will only degrade performance—every superfluous token dilutes attention and introduces noise. Hierarchical memory, the self-evolution mechanism, the minimal tool set, and browser-level page compression are all manifestations of this unified principle, and are detailed in the subsections that follow.

3.2 Core Technical Design

Guided by the principle of context information density maximization, GA's technical design spans four tightly coupled dimensions: memory organization, experience evolution, tool design, and browser control. These four dimensions are mutually reinforcing. The memory system governs the quality of knowledge admitted into the context; the self-evolution mechanism continuously compresses the token cost of execution; the minimal tool set controls the prompt overhead of tool definitions themselves; and browser control targets the high-noise scenario of web interaction with specialized page-level compression.

3.2.1 Hierarchical Memory System

The memory system constitutes the first line of defense for context information density optimization. GA employs a four-layer memory architecture that, through hierarchical distillation and taxonomy-based management, ensures only the highest-density knowledge enters the context window. The governing principle is: retain only the critical information that the model cannot derive through reasoning alone, so as to alter model behavior at minimal token cost.

L0: Meta-Rule Layer. The L0 layer stores the system's core behavioral norms and adheres to a strict minimalism principle. Each meta-rule is highly condensed—for example, the action-verification principle is expressed in a single sentence: "Only information that has been empirically verified through actual execution may be written to long-term memory," thereby preventing hallucination contamination at the root. These rules are loaded into the System Prompt at every agent startup, establishing foundational behavioral guidance at an extremely low token cost.

L1: Index Layer. The L1 layer provides rapid localization of relevant memories through a memory taxonomy. It stores only metadata—file paths, brief descriptions, and applicability conditions—rather than full content, avoiding unnecessary loading of complete memory entries. Before executing a task, the agent first consults the L1 index and selectively loads only the memories genuinely needed into the context. Even as the memory store grows large, the context contains only task-relevant information, achieving precise retrieval with minimal token expenditure. In practice, the L1 index is maintained under a hard limit of 30 lines (fewer than 1,000 tokens), ensuring that the retrieval overhead remains negligible regardless of the total memory volume.

L2: Fact Store. The L2 layer stores verified environmental information, organized thematically and condensed through progressive summarization. Only incremental information that the model cannot infer on its own is retained—project directory structures, API endpoints, configuration parameters, and the like. Every fact is validated through tool execution before being committed, ensuring reliability. Topic-based organization prevents information fragmentation and improves retrieval efficiency.

L3: Task-Level SOP Layer. The L3 layer crystallizes long-term memory into reusable Skills: highly refined records of key steps and caveats distilled from multiple task executions. Only the portions where the model is prone to error or cannot easily reason—exception handling sequences, tool invocation orderings, and common

failure avoidance strategies—are included. SOP refinement is carried out automatically through the Reflect mode, transforming raw experience into reusable knowledge that guides future tasks at minimal prompt cost.

Working Memory and Update Mechanisms. Working memory further optimizes context density by storing temporary information relevant to the current task. At each reasoning turn, an anchor prompt injects a `[WORKING MEMORY]` block containing the current turn number, stored key information from the session, and pointers to related SOPs. Upon task completion, a distillation step extracts valuable information and writes it to L2 or L3, after which working memory is cleared to prevent context pollution in subsequent sessions. Long-term memory updates employ two complementary strategies: precision editing, which modifies only the changed portions, and reflective distillation, which compresses accumulated experience. Together, these ensure the memory system is continuously compressed and optimized over time, steadily reducing prompt cost.

3.2.2 Reflection-Driven Self-Evolution

The self-evolution mechanism is the core driver behind GA's continuous improvement in token efficiency. Its central insight is that experience should be progressively compressed from natural language into directly invocable code, so that complex tasks can be completed with the fewest tokens possible. The system distills experience through the Reflect mode, consolidates high-density knowledge into SOPs and then into code, and finally registers the results as reusable Skills, realizing a self-driven evolutionary cycle.

Stage 1: Experience Distillation into SOPs. Through scheduled Reflect-mode sessions, the system autonomously analyzes task execution logs, focusing on the highest-density segments: effective strategies from successful tasks and recurring errors from failed ones. The agent extracts key steps and caveats from multi-turn tool calls and LLM reasoning traces, compressing lengthy execution trajectories into L3 SOPs. Only the portions where the model is error-prone or unable to reason independently are retained; routine steps and information that can be reconstructed are omitted, achieving information compression and knowledge distillation.

Stage 2: SOP Codification. For tasks that have been validated multiple times and exhibit a highly structured execution pattern, the system evaluates whether the corresponding SOP can be implemented as a Python script. For fixed-step, highly parameterized tasks—data transformation, batch file processing, periodic API calls—codification compresses what originally required multiple rounds of reasoning and tool invocation into a single `code_run` execution, yielding an order-of-magnitude reduction in token consumption.

Stage 3: Code Registration as Skills. Codified scripts are registered as Skills in the L3 layer and the L1 index is updated accordingly. When the agent subsequently encounters a similar task, it can invoke the relevant Skill directly without re-deriving the solution. The system continuously monitors the effectiveness of SOPs and Skills, optimizing underperforming workflows—simplifying steps, reordering tool calls, and updating caveats—so that execution efficiency and information density improve over time.

These three stages form a closed loop: reflection distills experience → SOPs accumulate → codification lowers execution cost → Skills support exploration of new tasks. Through this mechanism, GA achieves a steadily decreasing token cost for recurring task types as experience accumulates, while the freed token budget is redirected toward more complex tasks.

3.2.3 Minimal Atomic Tool Set

Tool definitions must be injected into the context on every LLM call, making tool design a direct determinant of context information density. To minimize the overhead introduced by tool schemas, GA adopts a minimal atomic tool set comprising only seven base tools, achieving complex task completion through principled composition. Table 1 summarizes the complete set.

Table 1 The seven atomic tools in GenericAgent.

Tool	Description
`code_run`	Execute Python/Shell scripts with timeout control and abort signals.
`file_read`	Read files by line range with keyword-based location support.
`file_write`	Create or overwrite files.
`file_patch`	Precise local replacement via unique old/new content pairs; forces `file_read` confirmation on mismatch.
`web_scan`	Extract the current browser page as simplified HTML, returning compressed Markdown and a tab list.
`web_execute_js`	Inject and execute JavaScript in the user's browser, capturing return values and DOM changes automatically.
`ask_user`	Synchronous blocking request for user input; the Agent Loop immediately exits the current turn upon invocation.

GA's tool design adheres to an atomicity principle: each tool provides an irreducible primitive capability—code execution, file read/write/patch, web content retrieval, browser-side execution, and user interaction. This minimal yet complete set covers every essential pathway for agent–environment interaction while avoiding the redundancy inherent in domain-specific tools.

The minimal tool set yields two direct benefits. First, it **significantly reduces tool-description overhead** in the context. In multi-turn interaction scenarios, tool definitions are repeatedly injected; fewer tools translate into a sustained reduction in cumulative token consumption across turns. Second, it **lowers the model's decision complexity**. A small, clearly bounded tool set reduces selection ambiguity, enabling the model to make more stable tool-call decisions and thereby reducing errors and retries.

In terms of expressiveness, GA does not rely on specialized tools to extend functionality but instead achieves complex operations through atomic-tool composition. File updates, for instance, follow a "read–modify–write" pattern; web data extraction is accomplished via "content retrieval–structural analysis–browser execution." This approach trades a small number of additional steps for a substantial gain in overall context efficiency. External capabilities such as MCP integrations are encapsulated as Skills that are invoked indirectly through the atomic tools and loaded into the context only on demand. Memory management is likewise implemented through sub-agents that operate the file-oriented tools, avoiding the introduction of additional tool types. Tool interfaces are kept minimal, retaining only the parameters strictly necessary, further reducing invocation complexity and token overhead.

3.2.4 Context-Compressed Browser Control

Browser interaction is one of the most severe sources of information redundancy in agent systems. Conventional approaches typically return the full web page content, including vast amounts of task-irrelevant structure, styling, and script information, resulting in extremely low context information density. GA redefines the core objective of browser control as minimum necessary information retrieval, systematically compressing page content, interaction traces, and execution paths to prevent low-value information from entering the context.

Content Layer: Simplified HTML. GA introduces a simplified HTML (simp HTML) mechanism that performs structural parsing and filtering of web pages, retaining only semantic information and key structural elements before converting them into a compact Markdown representation. The process operates in two stages: a first-stage JavaScript-side DOM analysis that classifies every node by relevance (e.g., primary content, secondary content, or covered/redundant), followed by a second-stage Python-side cleanup using BeautifulSoup that strips residual noise. Compared with raw HTML, this method eliminates styling, scripts, and redundant nesting, dramatically reducing token overhead while preserving informational completeness. Large lists are detected and automatically truncated to a representative sample (e.g., three items), and a smart budget-allocation mechanism ensures that token consumption stays within predefined limits. Hidden

elements are expanded on demand, enabling the agent to access potentially critical information that would otherwise be lost due to page-visibility constraints.

Interaction Layer: Incremental DOM Updates. Rather than returning the full page after every operation, GA performs incremental updates based on DOM changes, extracting only the content that was added or modified by the most recent action. This strategy avoids the repeated transmission of large volumes of unchanged information during multi-step interactions, keeping the context size stable as the task progresses and significantly improving the efficiency of long-chain browser workflows.

Execution Layer: TMWebDriver. The TMWebDriver component provides stable browser control through a dual-channel communication architecture: a WebSocket channel (port 18765) for real-time bidirectional messaging and an HTTP long-polling channel (port 18766) for reliable delivery, both interfacing with a Chrome extension. Adaptive communication and multi-tab parallel execution reduce unnecessary interaction rounds. The system further leverages the Chrome DevTools Protocol (CDP) to access network requests directly, allowing certain tasks to bypass page parsing entirely and retrieve results at a lower information-transfer cost.

Taken together, GA transforms browser control from "page-level information retrieval" into "task-relevant information extraction." Through the coordinated design of content compression, incremental updates, and execution-layer optimization, the system effectively maximizes context information density and interaction efficiency in web-based scenarios.

4 Conclusion

We have presented GenericAgent (GA), a general-purpose LLM agent system whose architecture is organized around a single guiding principle: context information density maximization. Every design decision—from the L0–L3 hierarchical memory that loads only task-relevant information on demand, through the reflection-driven self-evolution pipeline that compresses experiential knowledge into increasingly compact representations, to the seven minimal atomic tools and the simplified HTML browser control—serves to ensure that each token in the finite context window carries maximal value for the current reasoning step. Together, these four contributions form a coherent system that enables sustained, long-horizon autonomous operation across interactive, task-driven, and event-triggered scenarios.

A key insight underlying GA is that self-evolution operates entirely at inference time, requiring no fine-tuning of the base model. By progressively transforming free-form experience into structured SOPs, then into executable code, and ultimately into reusable skills, the system achieves order-of-magnitude reductions in the number of tokens needed to represent recurring operational knowledge, directly realizing the density maximization objective.

Several limitations merit discussion. First, the effectiveness of GA is ultimately bounded by the reasoning and instruction-following capabilities of the underlying language model; advances in base model quality translate directly into improved agent performance, but current ceilings remain binding for the most complex tasks. Second, the hierarchical memory taxonomy, while general in design, requires initial configuration effort to align with new application domains. Third, the self-evolution mechanism yields the most pronounced gains for tasks exhibiting regular, repeatable structure; highly novel or one-shot tasks benefit less from experience compression. Finally, the browser control subsystem assumes the availability of a Chrome extension for DOM observation, which introduces a deployment dependency.

Looking ahead, we identify several promising directions for future work. Multi-agent collaboration with shared hierarchical memory pools could enable teams of GA instances to pool experience and divide labor on complex workflows. Cross-task knowledge transfer—automatically migrating skills and SOPs across related but distinct domains—would further amplify the returns of self-evolution. Formal evaluation metrics for context information density, beyond proxy measures such as token counts, are needed to rigorously benchmark density-driven agent designs. Finally, integration with emerging agent communication protocols would allow GA to interoperate seamlessly within broader multi-agent ecosystems.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems, volume 35, pages 24824–24837, 2022.

[3] OpenAI. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[4] Anthropic. Claude code. https://www.anthropic.com/product/claude-code, 2025.

[5] OpenAI. Introducing codex. https://openai.com/index/introducing-codex/, 2025.

[6] Xingyao Wang, Boxuan Jiang, Ziniu Lu, Yufan Liu, Abishek Sridhar Li, Bolun Shi, Jiannan Fang, Rithvik Mohanty, Niklas Muennighoff Ponnapalli, Kaixuan Ren, et al. Openhands: An open platform for ai software developers as generalist agents. In International Conference on Learning Representations, 2025.

[7] Aditya Rajasekaran et al. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, September 2025.

[8] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. arXiv preprint arXiv:2310.08560, 2023.

[9] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

[10] Yichao Shen et al. Manus: From mind to machine. arXiv preprint arXiv:2505.02024, 2025.

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36:8634–8652, 2023.

[12] Significant Gravitas. Autogpt. https://github.com/Significant-Gravitas/AutoGPT, 2023.

[13] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In International Conference on Learning Representations, 2024.

[14] Cognition Labs. Devin: Ai software engineer. https://cognition.ai/blog/introducing-devin, 2024.

[15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao Narasimhan, and Karthik. Swe-agent: Agent-computer interfaces enable automated software engineering. Advances in Neural Information Processing Systems, 37, 2024.

[16] Xingyao Wang et al. Openhands software agent sdk. arXiv preprint arXiv:2511.03690, 2025.

[17] Xingyao Wang et al. Openclaw-rl: Training agents via interaction. arXiv preprint arXiv:2603.10165, 2026.

[18] Wujiang Xu et al. A-mem: Agentic memory for llm agents. Advances in Neural Information Processing Systems, 38, 2025.

[19] Zhiyuan Gao et al. A survey of self-evolving agents. Transactions on Machine Learning Research, 2026.

[20] Xuechen Wu et al. Evolver: Self-evolving llm agents through experience-driven lifecycle. arXiv preprint arXiv:2510.16079, 2025.

[21] Anonymous. Flex: Continuous agent evolution via forward learning from experience. arXiv preprint arXiv:2511.06449, 2025.

[22] Anonymous. Agentevolver: Towards efficient self-evolving agent system. arXiv preprint arXiv:2511.10395, 2025.

[23] Tianyu Cai et al. Building self-evolving agents via experience-driven lifelong learning. arXiv preprint arXiv:2508.19005, 2025.

[24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, volume 36, 2023.

[25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334, 2023.

[26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolbench: An open platform for training, serving, and evaluating large language model tool-use. arXiv preprint arXiv:2305.16504, 2023.

[27] Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/research/building-effective-agents, December 2024.

[28] Anthropic. Developing a computer use model. https://www.anthropic.com/research/developing-computer-use, October 2024.

[29] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, 2024.

[30] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In Advances in Neural Information Processing Systems, volume 36, 2023.

Appendix

.1 System Prompt

The following is the complete system prompt injected into every LLM call within GenericAgent. The original prompt is written in Chinese; we provide a faithful English translation below.

GenericAgent System Prompt (Translated)

 # Role: Physical-Level Omnipotent Executor

You possess physical operation permissions for file read/write, script execution, user browser JS injection, and system-level intervention. Never deflect with "cannot operate" — do not speculate; use tools to probe.

Action Principles

Before calling any tool, reason inside `<thinking>`: current stage, whether the previous result matches expectations, and next-step strategy.

- **Probe First:** On failure, gather sufficient information first (logs / status / context), store key information in working memory, then decide whether to retry or switch plans. For irreversible operations, ask the user first.
- **Failure Escalation:** 1st failure → read error and understand the cause; 2nd failure → probe environment state; 3rd failure → deep analysis then switch plan or ask user. Repeating the same operation without new information is forbidden.

.2 Tool Definitions

GenericAgent exposes exactly seven atomic tools to the LLM. Table 2–8 present their complete schemas as provided in the system prompt.

Table 2 Tool schema: `code_run`

code_run — Code executor. Prefer Python. No concurrent calls.
Prefer code in fenced blocks in reply body to avoid escaping. No hardcoding bulk data.

Parameter	Type	Description
`script`	string	*[Mutually exclusive]* Never use this param when using reply code block.
`type`	enum	Code type: `python` | `powershell`. Default: `python`.
`timeout`	integer	Timeout in seconds. Default: 60.
`cwd`	string	Working directory; defaults to current working directory.

Table 3 Tool schema: `file_read`

file_read — Read file. Read before modify for latest context and line numbers.

Parameter	Type	Description
`path`	string	Relative or absolute file path.
`start`	integer	Start line number (1-based). Default: 1.
`count`	integer	Number of lines to read. Default: 200.
`keyword`	string	*[Optional]* If provided, returns first match (case-insensitive) with context.
`show_linenos`	boolean	Show line numbers. Default: `true`.

Table 4 Tool schema: `file_patch`

file_patch — Replace unique `old_content` with `new_content`. Exact match required (whitespace/indentation). On failure, use `file_read` to recheck.

Parameter	Type	Description
`path`	string	File path.
`old_content`	string	Original text block to replace (must be unique in the file).
`new_content`	string	New content. Supports `{{file:path:startLine:endLine}}` to reference file lines; auto-expanded.

Table 5 Tool schema: `file_write`

file_write — Create / overwrite / append files. Only for large edits. Content in `<file_content>` tags or reply code blocks. Supports `{{file:...}}` references.

Parameter	Type	Description
`path`	string	File path.
`mode`	enum	Write mode: `overwrite` | `append` | `prepend`. Default: `append`.

Table 6 Tool schema: `web_scan`

web_scan — Get simplified HTML and tab list. Removes hidden / floating / covered elements. Call after switching pages.

Parameter	Type	Description
`tabs_only`	boolean	Show tab list only, no HTML. Default: `false`.
`switch_tab_id`	string	*[Optional]* Tab ID to switch to.
`text_only`	boolean	Plain text only, no HTML. Default: `false`.

Table 7 Tool schema: `web_execute_js`

web_execute_js — Execute JS to control browser. No guessing. Act accurately to reduce `web_scan` calls. Put code in fenced JS blocks in reply body.

Parameter	Type	Description
`script`	string	*[Mutually exclusive]* JS code or script path. Never use when using reply code block.
`save_to_file`	string	File path; only for long results. Default: empty.
`no_monitor`	boolean	Skip page-change monitoring (saves 2–3 s). Only for reads, not for page actions. Default: `false`.

Table 8 Tool schema: `ask_user`

ask_user — Interrupt task to ask user when needing decisions, extra info, or facing unresolvable blockers.

Parameter	Type	Description
`question`	string	Question for the user.
`candidates`	array[string]	*[Optional]* Quick-select choices for the user.

.3 Memory System Templates

.3.1 L1 Global Memory Insight Template

The L1 layer serves as the ultra-compact navigation index (≤30 lines) that is injected into every LLM call. The following is the template structure, translated from Chinese.

L1 Global Memory Insight Template (Translated)

```
# [Global Memory Insight]
Browser automation: web_scan / web_execute_js direct call
| Special: tmwebdriver_sop (file upload / image search /
PDF blob / element coords / Cookie extraction incl.
HttpOnly / cross-origin iframe / CDP / cross-tab /
background tab ops)
Keyboard-mouse simulation: ljqCtrl_sop+.py
(Windows only; no pyautogui; activate window first)
Scheduled tasks: scheduled_task_sop
(reports -> sche_tasks/done/)
| Completely independent from autonomous tasks
Autonomous exploration: autonomous_operation_sop
(reports -> temp/autonomous_reports/history.txt)
| Completely independent from scheduled tasks
Mobile control: adb_ui.py
When needed: read L2 or ls ../memory/ to find L3
L0 (META-SOP): memory_management_sop
L2: (currently empty)
L3: web_setup_sop | autonomous_operation_sop |
scheduled_task_sop | ljqCtrl_sop+.py |
tmwebdriver_sop | subagent_sop | plan_sop |
mem_scanner.py | adb_ui.py
[RULES]
1. Search first: use Google (must be web) for info;
os.listdir for project internals; never guess paths
2. Cross-verify: never trust search snippets;
verify numerical values on detail pages
3. Coding safety: always read source before modifying;
use sys.path.append for memory imports
4. Closed loop: physically verify after simulation;
request intervention after 3 failures
5. Processes: never unconditionally kill python
(would kill self); use precise PIDs;
never use os.kill to check liveness
6. Windows: prefer enumerating windows over OCR
for GUI state - much faster
7. Physical red line: cwd uses ./;
after cwd is set, never use ../ in code;
use absolute paths instead
8. Web JS: get it right the first time;
use native setters + event chains for input;
check disabled before clicking; mind quote escaping;
```

```
if scan returns empty, scan again or use innerText
9.  SOP: read cached hard params before execution;
never rely on memory; if utils exist, use them;
read plan_sop for complex long-horizon tasks
10.  Read plan_sop to enter planning mode for tasks
that require or mention planning
```

.3.2 L0 Memory Management SOP -- Core Axioms

The L0 meta-rules govern all memory writes across the hierarchy. The following are the four core axioms (highest priority), translated from the original Chinese specification.

L0 Core Axioms (Translated from `memory_management_sop.md`)

Axiom 1 --- Action-Verified Only. Any information written into L1/L2/L3 must originate from a successful tool-call result (e.g., shell execution succeeded, `file_read` confirmed content exists, code ran without error). It is strictly forbidden to write the model's "inherent knowledge," "speculative reasoning," "unexecuted plans," or "unverified hypotheses" as facts.
Motto: No Execution, No Memory.

Axiom 2 --- Sanctity of Verified Data. Any action-verified configuration, pitfall guide, or critical path **must never be discarded** during refactoring or garbage collection. Text may be compressed and entries may be migrated across layers (e.g., from L2 to L3), but the accuracy and traceability of the information must be preserved. When modifying memory, exercise extreme caution; prefer small patches over overwrites.

Axiom 3 --- No Volatile State. It is strictly forbidden to store data that changes frequently over time or across sessions. Examples: current timestamps, temporary session IDs, running PIDs, specific absolute paths, connected device information.

Axiom 4 --- Minimum Sufficient Pointer. Upper layers retain only the shortest identifier needed to locate the lower layer. One extra word is redundancy.

In addition, the L0 SOP defines the **information classification decision tree** that determines the layer assignment for any new piece of knowledge:

Information Classification Decision Tree

```
"Where should this information go?"

Is it an 'environment-specific fact'?
(IP, non-standard path, credential, ID, API key, etc.
 -- model cannot generate accurately via zero-shot)
  |-- YES -> L2 (global_mem.txt)
  |          then -> by frequency:
  |            high-freq -> L1 first tier (key->value)
  |            low-freq  -> L1 second tier (keyword only)
  |
  '-- NO
       |
       Is it a 'universal operational pattern'?
       (global pitfall guide, debugging method,
        not task-specific)
```

```
|-- YES -> L1 [RULES] (one compressed sentence)
|
'-- NO
     |
     Is it 'task-specific technique'?
     (succeeded only after hard attempts,
      reusable in the future)
     |-- YES -> L3 (../memory/ dedicated SOP
     |               or script)
     |
     '-- NO -> 'Common knowledge' or 'redundant':
               strictly forbidden to store; discard
```

.4 Case Studies

We provide detailed case studies demonstrating GenericAgent's self-evolution process in the supplementary materials.